

Service Accounts in Kubernetes

Identity for your workloads — what they are, how RBAC fits, and where they're used

What is a Service Account?

A **service account** is an identity used by processes running inside pods to authenticate to the Kubernetes API server (and sometimes to external systems). Think of it as a "user account" for your applications — whereas regular user accounts are meant for humans.

When a pod needs to talk to the Kubernetes API — for example to list other pods, read a ConfigMap, or watch for changes — it must prove who it is. The service account provides that identity along with a token the pod uses to authenticate. Permissions are then granted to it through RBAC.

User accounts are for humans, managed outside the cluster (certificates, OIDC). **Service accounts** are for in-cluster workloads, managed as Kubernetes objects.

Key Things to Know

- **Every namespace** gets a default service account automatically. If you don't specify one, pods use it (with very limited permissions).
- **Tokens are mounted automatically** into each pod at `/var/run/secrets/kubernetes.io/serviceaccount/`.
- In modern Kubernetes (1.22+), these are **short-lived, auto-rotated** projected tokens — not the old permanent secret-based tokens.
- You attach a service account to a pod via the `serviceAccountName` field in the pod spec.

The Three Pieces That Work Together

A service account on its own can authenticate but can't *do* anything until RBAC grants it permissions. Three objects combine:

ServiceAccount	Role	RoleBinding
The identity (<i>who</i>)	The permissions (<i>what is allowed</i>)	The glue (<i>who gets what</i>)

Mental model: `ServiceAccount --(RoleBinding)--> Role`
A Role does nothing until a RoleBinding attaches it to a subject.

A Complete Working Example

1. The ServiceAccount (the identity):

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: default
```

2. The Role (the permissions):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]          # "" = core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

3. The RoleBinding (the glue):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

4. The Pod that uses it:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  serviceAccountName: my-app-sa
  containers:
  - name: app
    image: my-app:latest
```

Scope: Role + RoleBinding are **namespaced** — they only grant permissions within one namespace. For cluster-wide access (or cluster-scoped resources like nodes), use **ClusterRole** + **ClusterRoleBinding** instead. Same structure, whole-cluster reach.

Common Use Cases

- **CI/CD pipelines**

Tools like Jenkins, GitLab CI, ArgoCD and GitHub Actions need API access to deploy into the cluster. Give the pipeline a dedicated service account and use RBAC to scope it — e.g. "can create/update deployments, but cannot delete secrets."

- **Monitoring & logging**

Prometheus, Datadog agents and Fluentd scrape data across the cluster, so they need to list/watch pods, nodes and metrics. A read-only service account (get/list/watch) is enough — never grant write permissions here.

- **Cloud resource access**

The biggest use case. Instead of hardcoding cloud credentials in a pod, bind the service account to a cloud IAM role: **IRSA** on EKS, **Workload Identity** on GKE and AKS. The pod then accesses S3, Secrets Manager or databases with no keys stored on disk.

- **Operators & controllers**

Custom controllers (cert-manager, external-dns) continuously watch and modify cluster state, so they need a capable, carefully-scoped service account.

- **Service mesh identity**

Tools like Istio use the service account as the workload identity for mTLS — securing pod-to-pod authentication.

- **Jobs & CronJobs**

Scheduled automation such as backups and cleanup tasks uses a service account to act on the cluster on a schedule.

Best Practices

1. Give each workload its **own** service account — don't share the default SA.
2. Apply **least privilege** — grant only the verbs and resources actually needed.
3. **Never** grant `cluster-admin` unless absolutely necessary.